© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_4

4. Bring Your Own Model: Self-Hosting with Ollama

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

This chapter guides you through the process of setting up a locally hosted large language models (LLMs)/small language models (SLMs), which you can use to test your applications.

Introduction

In Chapter 2, you learned how to integrate your applications with OpenAI APIs to drive results using prompt engineering. When using OpenAI APIs, there is always a case of vendor lock-in. In the world of software engineering, we have all been exposed to open-source alternatives that are best of breed. The more choice, the better in terms of developer friendliness, flexibility, and ability to choose what works best for you. There is also the cost aspect of it where you are charged on a per-usage basis for your requests. In this case, self-hosting in a cloud environment may make better sense in your context. Finally, there is the aspect of privacy. We all have concerns as to how our application data and user/customer data is being used by OpenAI. There are containerized versions of OpenAI plans that allow different customers to leverage the same APIs without them being used to train their internal models, so you always have to be cognizant of this. This is where applications like Ollama come into use to democratize Generative AI (Gen AI) hosting without being locked in or being dependent on a particular vendor.

In this chapter, you will learn about Ollama and what self-hosting an LLM means. You will install Ollama and use it to run an LLM on your local computer. All this does not make sense if you have to change your APIs and their request format since you have already integrated OpenAI APIs into your integration. Worry not, this is where the APIs exposed by Ollama come into picture. They are compliant with OpenAI APIs, meaning your application works as it is now with no change.

Self-Hosting LLMs

Running LLM models locally on your own hardware under self-hosting LLMs offers advantages including increased control over the models, more privacy, and less dependency on cloud services. Choosing the appropriate model comes first in order to get going.

Popular options for self-hosting are BERT or T5 in addition to GPT-based versions like GPT-2 or GPT-Neo. Given that larger models—such as GPT-3—demand far more powerful technology than smaller variants like GPT-2, the model size is a major consideration. Another important consideration is meeting the hardware needs. Usually, running LLMs efficiently calls for a high-performance GPU—especially for bigger models. For simple chores or smaller models, a good CPU could be enough. Regarding memory and storage, big models can call for significant resources; some models need tens or even hundreds of gigabytes of random access memory (RAM). For really big models, cloud services or specialized hardware—such as servers with many GPUs—may be required.

Locally running the model will require machine learning frameworks including TensorFlow, PyTorch, or Hugging Face Transformers, which provide the required infrastructure for model deployment. The model can be contained using Docker, therefore streamlining the installation and administration process. Furthermore, helping to lower memory use and raise the general model efficiency are optimization methods including mixed precision (FP16) and model quantization.

Quantization involves converting model weights from high-precision formats like 32-bit floats to lower-precision types such as 8-bit integers. For example, a weight of 0.865 in float32 might be approximated as 111 on the int8 scale, using a simple mapping function. While this introduces minor accuracy trade-offs, it significantly reduces the model's memory footprint and computational cost. We'll explore quantization in more detail with hands-on examples in a subsequent chapter.

Once the model is operational, you can expose its features by means of an API, therefore facilitating a simple interface with other programs or users. For inference jobs, a REST or gRPC API can be configured using well-known frameworks such as Spring or FastAPI or Flask. Creating a web app using tools like Streamlit or Gradio is a terrific way if you're seeking a more user-friendly interface.

Still, self-hosting has some expenses. Particularly if you use high-performance GPUs, initial hardware expenditures can be really large. Particularly in systems running continually, operational expenses like electricity for running the hardware and cooling systems can also mount up. Another continuous responsibility is maintaining the infrastructure supporting hardware and software. Having said that, the

main benefit of self-hosting is data privacy. By executing the model locally, you have complete control over the data and therefore lower the dangers connected with using outside cloud solutions. Strong security measures, like encryption and API key authentication, will be necessary, though, should you expose the model to the Internet via an API to guarantee the system is protected from harmful usage.

Self-hosting LLMs can be aided using several tools and frameworks. One such solution that streamlines the local operation of LLMs and provides an easy-to-use interface for model deployment and management is Ollama. Pretrained models and an inference API also come from Hugging Face, which would be helpful for local or cloud server deployment. Excellent substitutes for more costly models like GPT-3 and fit for self-hosting are open-source models including GPT-Neo, GPT-J, and GPT-2. Appropriate for implementation in resource-limited contexts, Meta's LLaMA models are another open-source solution that strikes performance and resource balance. That said, self-hosting big models presents difficulties including hardware restrictions, maintaining current software, and infrastructure scaling for several users or high-demand programs. Effectively running LLMs at scale depends on balancing these difficulties.

Ollama

Ollama is a platform and tool for running small and large language models locally on your own machine. It allows users to implement LLMs for tasks like text generation, summarization, and other AI-driven language tasks, without needing to rely on cloud-based services. This can offer benefits like increased privacy, faster response times, and the ability to work offline.

Ollama typically provides an easy interface to interact with models like GPT-style models, enabling you to integrate and experiment with these models in a more localized, controlled environment. It's designed to be simple to set up and use, and it supports various models that can be downloaded and run directly on our computer.

Privacy is one of the most compelling reasons to run models locally. When data never leaves your device or controlled infrastructure, the risk of it being intercepted, logged, or used for model training by third-party vendors is significantly reduced. This is especially critical for sensitive domains like healthcare, finance, and legal services. However, privacy is only as strong as the security mechanisms in place, and local deployments must be secured to ensure data protection.

To help safeguard against unauthorized access, Ollama can be configured with several best-practice security measures:

- Bind APIs to localhost by default, preventing remote access unless explicitly configured otherwise.
- If remote access is needed (e.g., across a local network or for team collaboration), network-level protections such as firewalls or VPNs should be used.
- Reverse proxies (e.g., NGINX or Traefik) can be deployed with HTTPS and basic auth or token-based authentication to protect public-facing endpoints.
- Ollama itself can integrate with authentication layers, and additional middleware can enforce rate-limiting and access control policies.

These mechanisms give developers confidence that their private data stays private, even when models are accessed via local APIs.

Alternatives to Ollama

While Ollama offers a streamlined and developer-friendly experience, it's not the only option available for self-hosting LLMs. Table 4-1 compares the different platforms.

Table 4-1 Ollama Alternatives

Platform	Highlights
LM Studio	Best GUI-based desktop app. Great for nondevelopers, with model discovery and a chat interface; model interaction remains local and private.
Text-generation-webui	A full-featured web interface supporting multiple backend engines (transformers, llama.cpp, etc.) and extensive customization options.
GPT4All	A beginner-friendly app focused on local chat experiences; highly polished desktop application.
LocalAI	A drop-in OpenAI API replacement enabling flexible local model serving for developers.
vLLM	High-throughput inference engine optimized for handling concurrent requests at scale.
OpenLLM	Provides OpenAI-compatible APIs, Docker/Kubernetes integrations, optimized inference backends, and enterprise deployment tooling.
Llama.cpp	Low-level C/C++ inference engine with fine-grained control, advanced quantization (2-bit to 8-bit), multi-hardware support, and OpenAI-compatible server via llama-server.

Platform	Highlights
Llamafile	Combines llama.cpp + model into a single portable executable; significantly faster prompt handling—30% to 500% speed gain over llama.cpp in many cases.

Why choose Ollama?

- It's exceptionally easy to set up with command-line simplicity.
- Great cross-platform support, especially on macOS.
- Active model support and default privacy-first behavior.

Why consider others?

- For a rich graphical interface, LM Studio or GPT4All may be better suited (especially for nontechnical users).
- For production-grade serving or high concurrency, OpenLLM or vLLM offer enhanced scalability and integrations.
- Developers needing deep backend flexibility or OpenAI API emulation might prefer LocalAI or text-generation-webui.

Installing Ollama

Ollama can be installed on macOS, Linux, and Windows. This section goes through the steps for each of them. Always make sure to check the Ollama downloads documentation page at **https://ollama.com/download** to refer to the latest information.

Installing Ollama on macOS:

Ollama provides DMG packages that can be downloaded and then installed. The steps are self-explanatory to follow through. The package can be downloaded by visiting **https://ollama.com/download/Ollama-darwin.zip**. It requires macOS 11 Big Sur or later. The package size is approximately around 180MB to download. Unzip it once downloaded.

As shown in Figure 4-1, Ollama starts up, and the system instructs you if you should move it to the Applications folder first and then start the installation process. In our case, you can choose Do Not Move and proceed. See Figure 4-2.

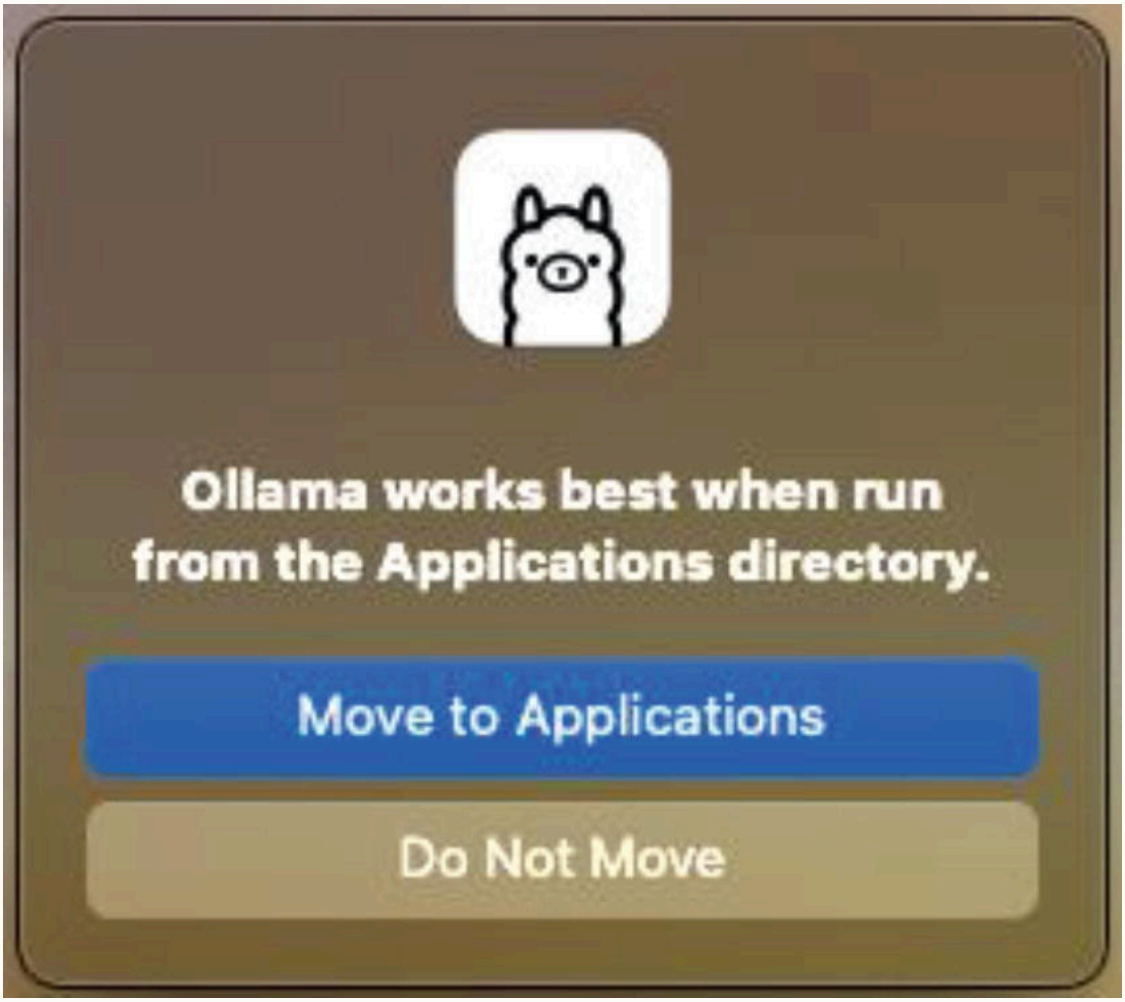


Figure 4-1 Starting Ollama for the first time after downloading

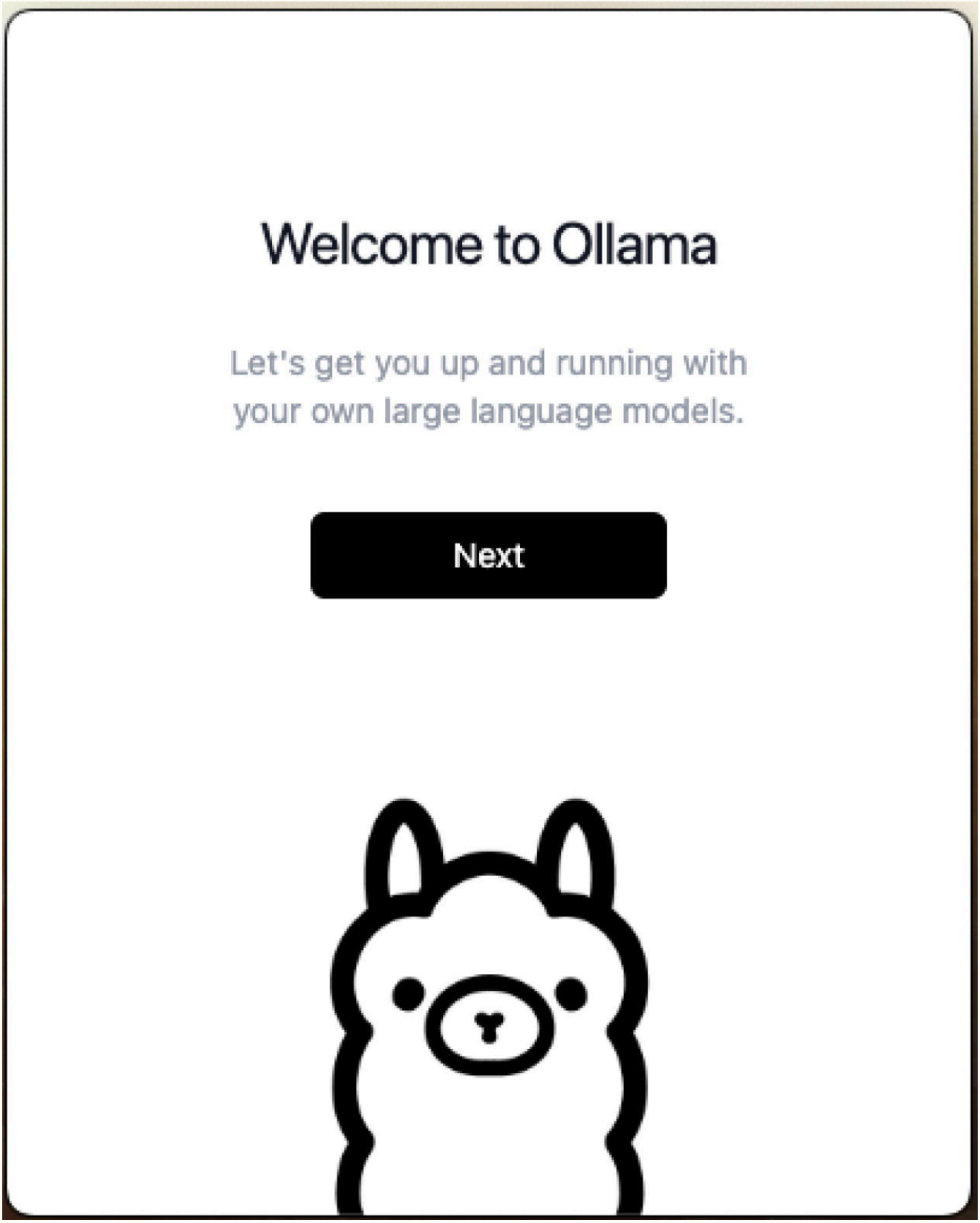


Figure 4-2 Starting Ollama

Ollama now starts up, and you can click Next to proceed to installing the command line, as shown in Figure 4-3. Once this is installed, you can open a terminal and run the ollama command to see output similar to what you saw in the "Installing Ollama in Linux" section.

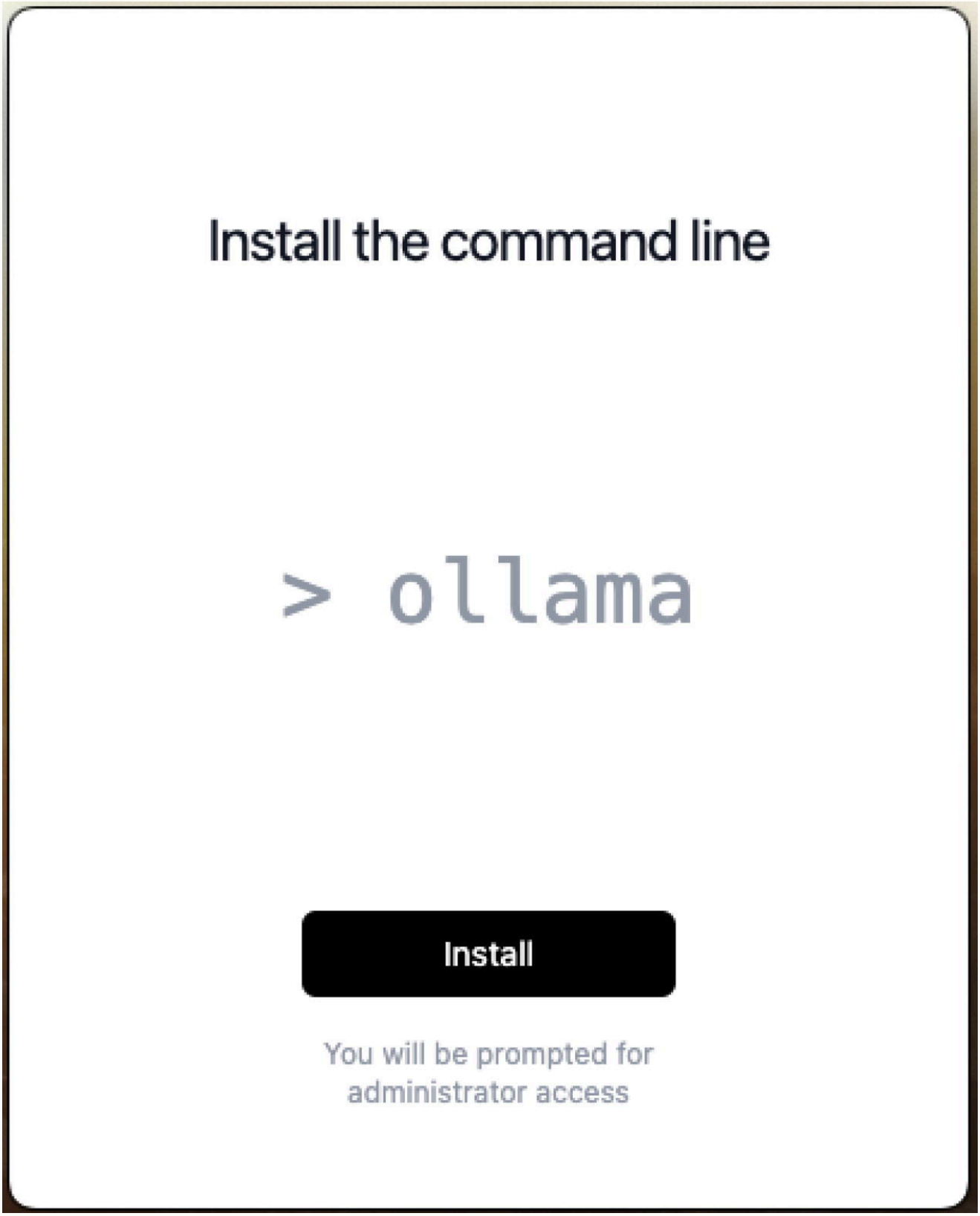


Figure 4-3 Installing Ollama

Installing Ollama in Windows:

Ollama provides Windows executable that can be downloaded and then installed. The steps are self-explanatory to follow through. The executable can be downloaded by visiting **https://ollama.com/download/OllamaSetup.exe**. The download size is approximately 745 MB, and the version downloaded is 0.5.7. Once it is downloaded, run the executable, which should guide you to Figure 4-4 to install Ollama.

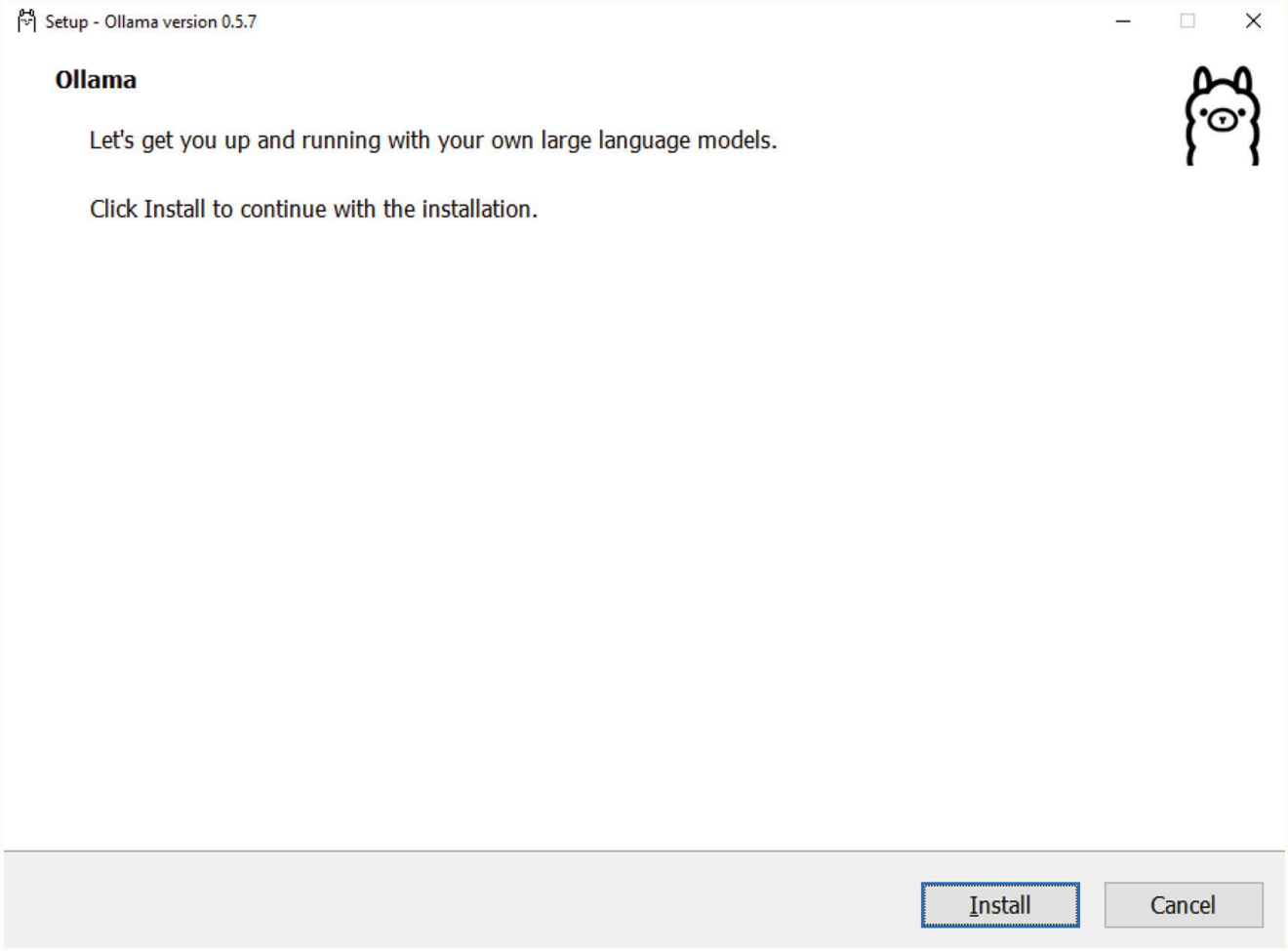


Figure 4-4 Ollama installation window

Proceed by clicking the Install button to start the installation, as shown in Figure 4-5.

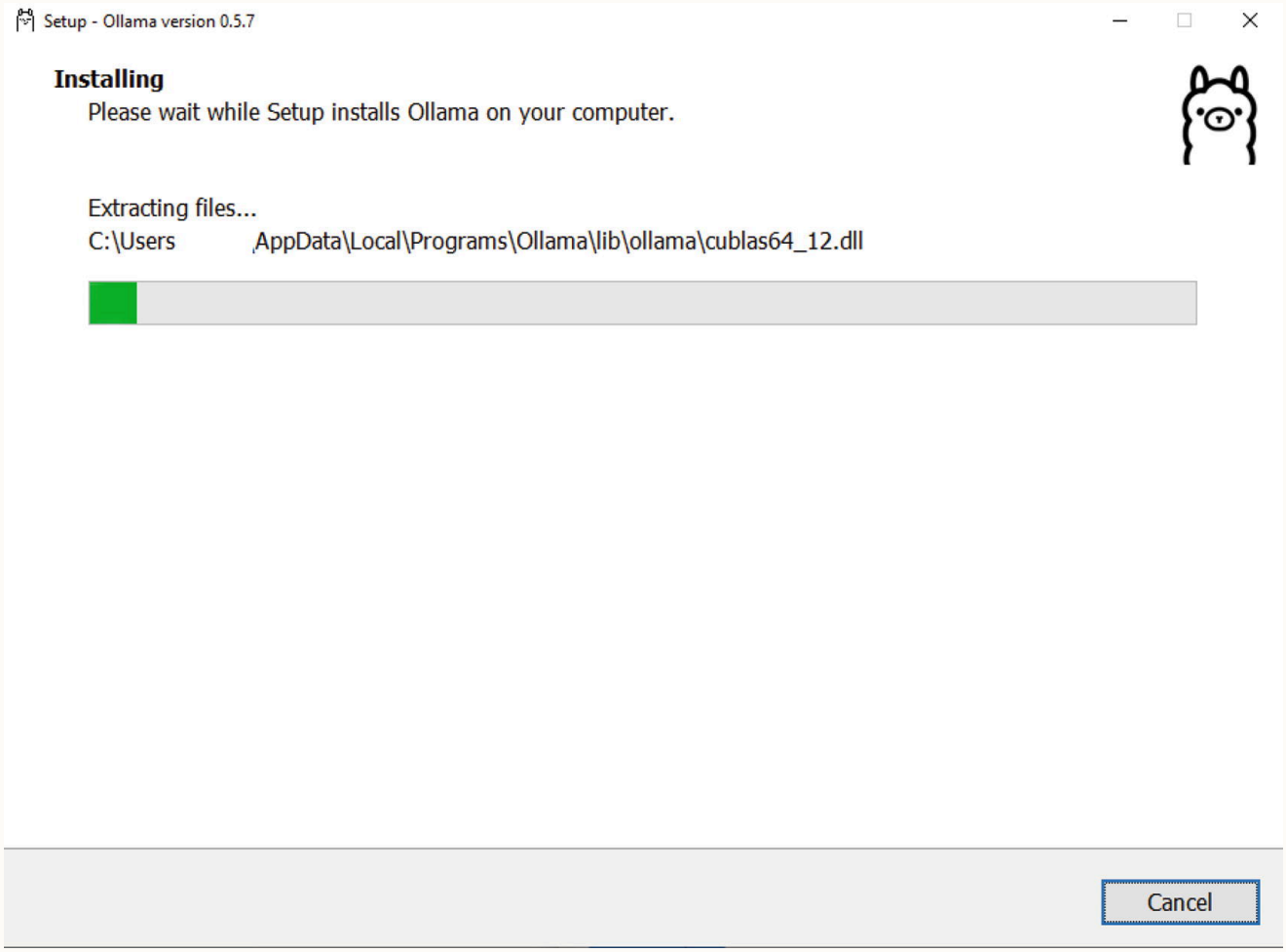


Figure 4-5 Installing Ollama

Once the installation is complete, an Ollama icon will pop up in the bottom-right task bar. Right-click it to see logs or exit Ollama. Open a command prompt or PowerShell and run the ollama command to see it in effect.

```
ollama
```

Output:

```
C:\Users\username>ollama
Usage:
 ollama [flags]
 ollama [command]
Available Commands:
 serve     Start ollama
 create    Create a model from a Modelfile
 show      Show information for a model
 run       Run a model
 stop      Stop a running model
 pull      Pull a model from a registry
 push      Push a model to a registry
 list      List models
 ps        List running models
 cp        Copy a model
 rm        Remove a model
 help      Help about any command
Flags:
 -h, --help    help for ollama
 -v, --version   Show version information
Use "ollama [command] --help" for more information about a command.
```

Installing Ollama in Linux:

Ollama can be installed in Linux by running the following curl command in a terminal. Make sure you have sudo permissions on the system since it will prompt for your password to install it to the /usr/local directory. It will then start downloading the Linux bundle specific to your system architecture, which in my case was amd64. It will try to detect the GPU to see if it can leverage it. If not, it works in CPU-only mode, which is good in this case.

```
curl -fsSL https://ollama.com/install.sh | sh
```

Output:

```
>>> Installing ollama to /usr/local
[sudo] password for <PLACEHOLDER_FOR_YOUR_SYSTEM_USERNAME>:
######################################################################## 100.0%
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.
```

As you can see in the output toward the end, it shares some information about the Ollama API being exposed at 127.0.0.1:11434. We will come back to this part in the “Exploring the APIs exposed by Ollama” section. This marks the completion of installing Ollama.

You can now type ollama in the terminal to see its usage and commands available.

```
ollama
```

Output:

```
Usage:
```

```
  ollama [flags]
  ollama [command]
Available Commands:
  serve     Start ollama
  create    Create a model from a Modelfile
  show      Show information for a model
  run       Run a model
  stop      Stop a running model
  pull      Pull a model from a registry
  push      Push a model to a registry
  list      List models
  ps        List running models
  cp        Copy a model
  rm        Remove a model
  help      Help about any command
Flags:
  -h, --help    help for ollama
  -v, --version   Show version information
Use "ollama [command] --help" for more information about a command.
```

Running a Locally Hosted LLM

Ollama provides hosting and running different open-source SLM/LLM models. The list keeps changing based on the recent developments, and it's quite fast. At this moment, visiting the models page at **https://ollama.com/search** shows the models in Figure 4-6 at first glance ordered by popularity.

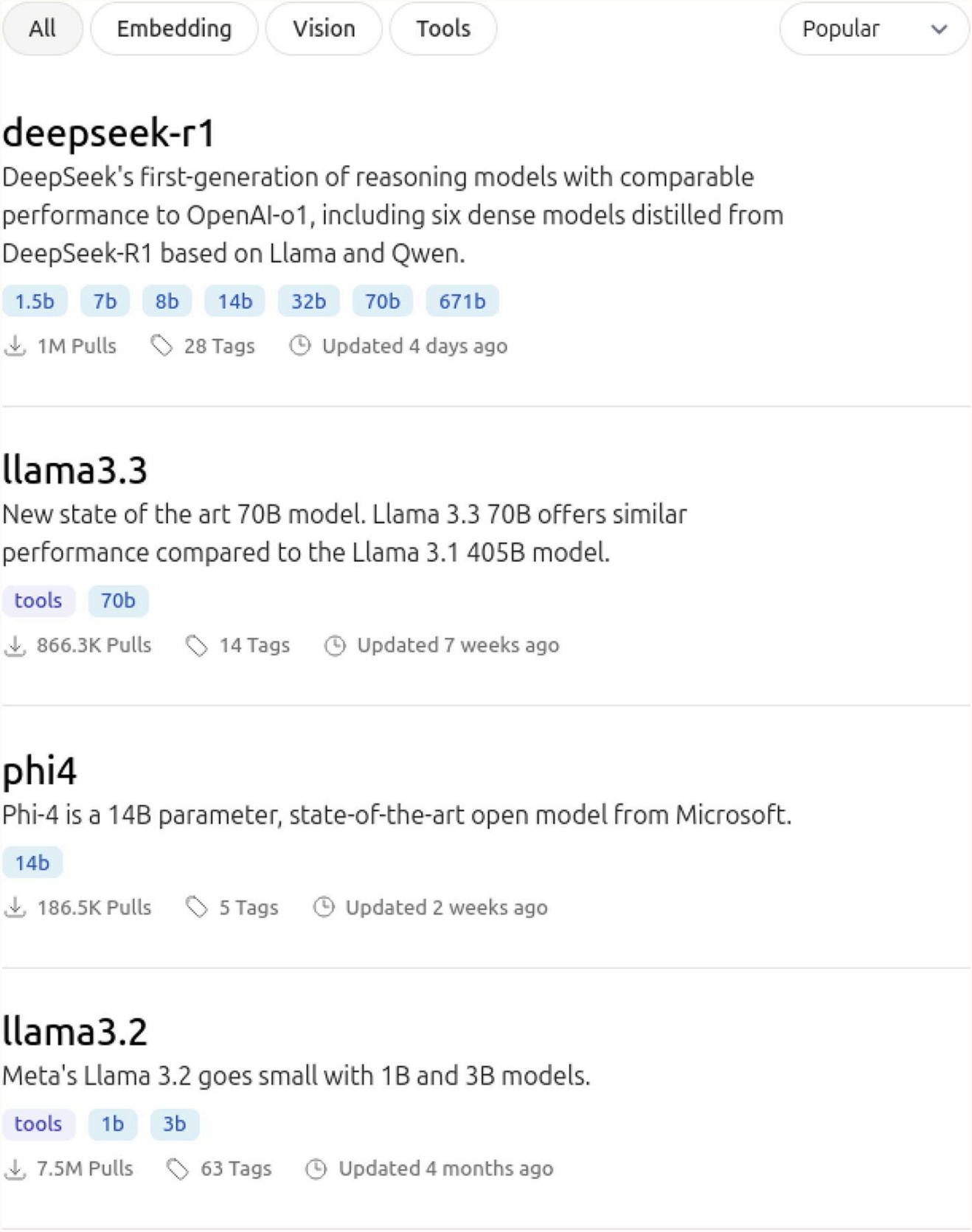


Figure 4-6 SLM/LLM models ordered by popularity

Let's explore the information shared here for a moment (see Figure 4-7).

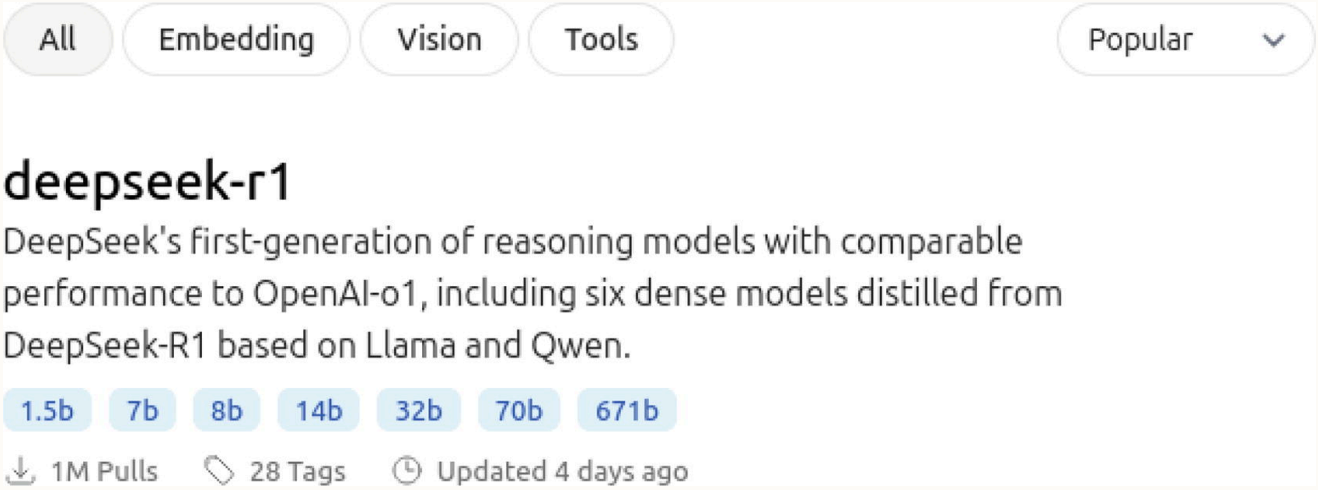


Figure 4-7 deepseek-r1 model information

As shown, the interface shares information about the category of models available, namely, All, Embedding, Vision, and Tools. You will learn more about these in future chapters. For the moment, you will work in the All category. On the right, you can order by either Popular or Newest models. For a particular model, it shares the name, which in this case is deepseek-r1, followed by a description of it, and then the different model sizes available for download and experimentation. At this moment, this has been pulled for installation approximately 1 million times and was updated last four days ago.

We will try the deepseek-r1 model of 1.5b size since it's small, fast, and quick to download and run on a local machine. On top of it, it is open-source, very efficient compared to OpenAI's o1 model, which is available only through a paid subscription and proprietary. Click the model you want to run, in this case deepseek-r1. It takes us to the library page of the model at https://ollama.com/library/deepseek-r1 (see Figure 4-8).

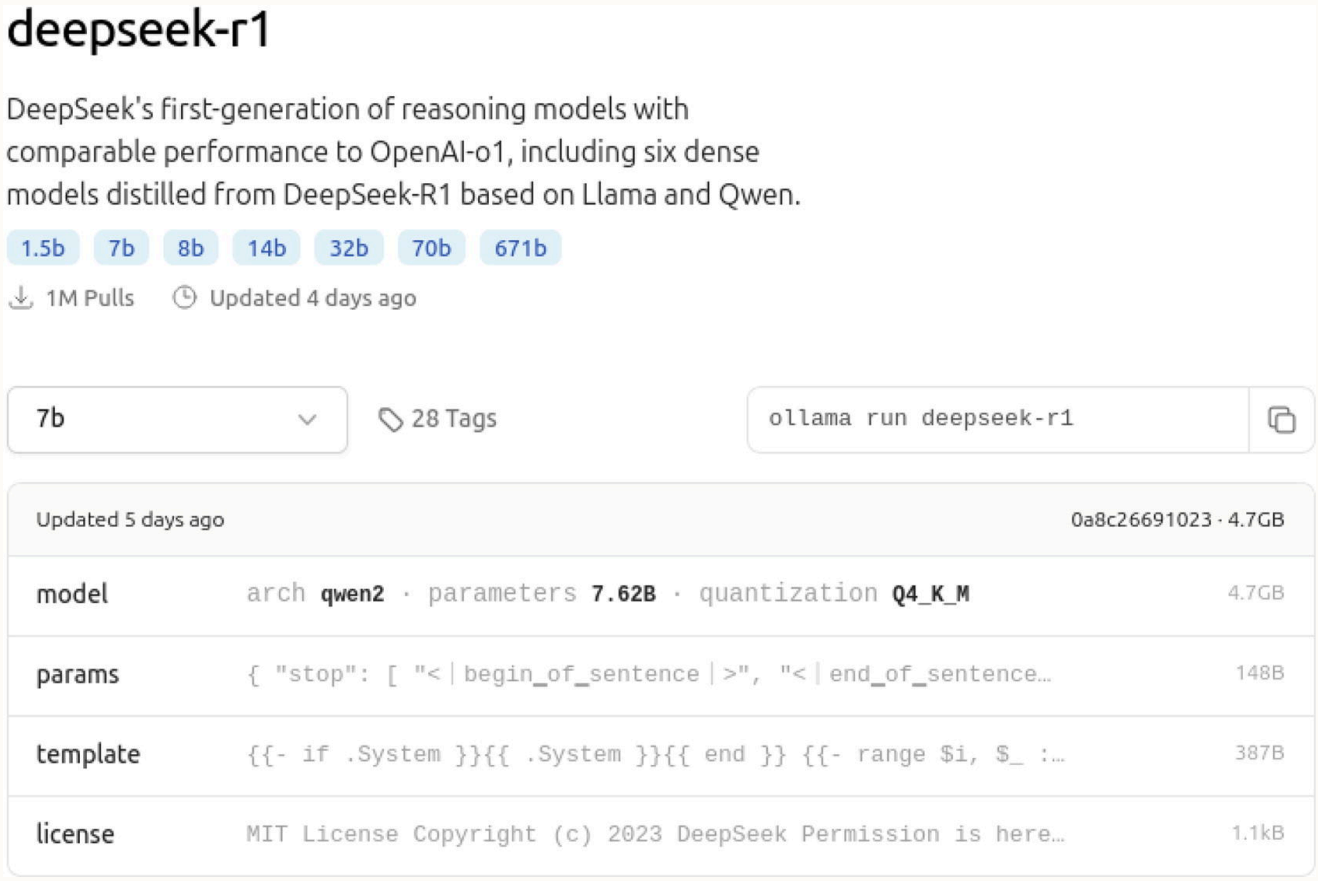


Figure 4-8 Model-specific information along with default model size and installation run command

Figure 4-9 shows some general information like the model and what architecture is based upon and the parameter size. You can click the drop-down to choose the specific model you want to run and see the command specific to it (see Figure 4-10).



Figure 4-9 deepseek-r1 1.5b size model information

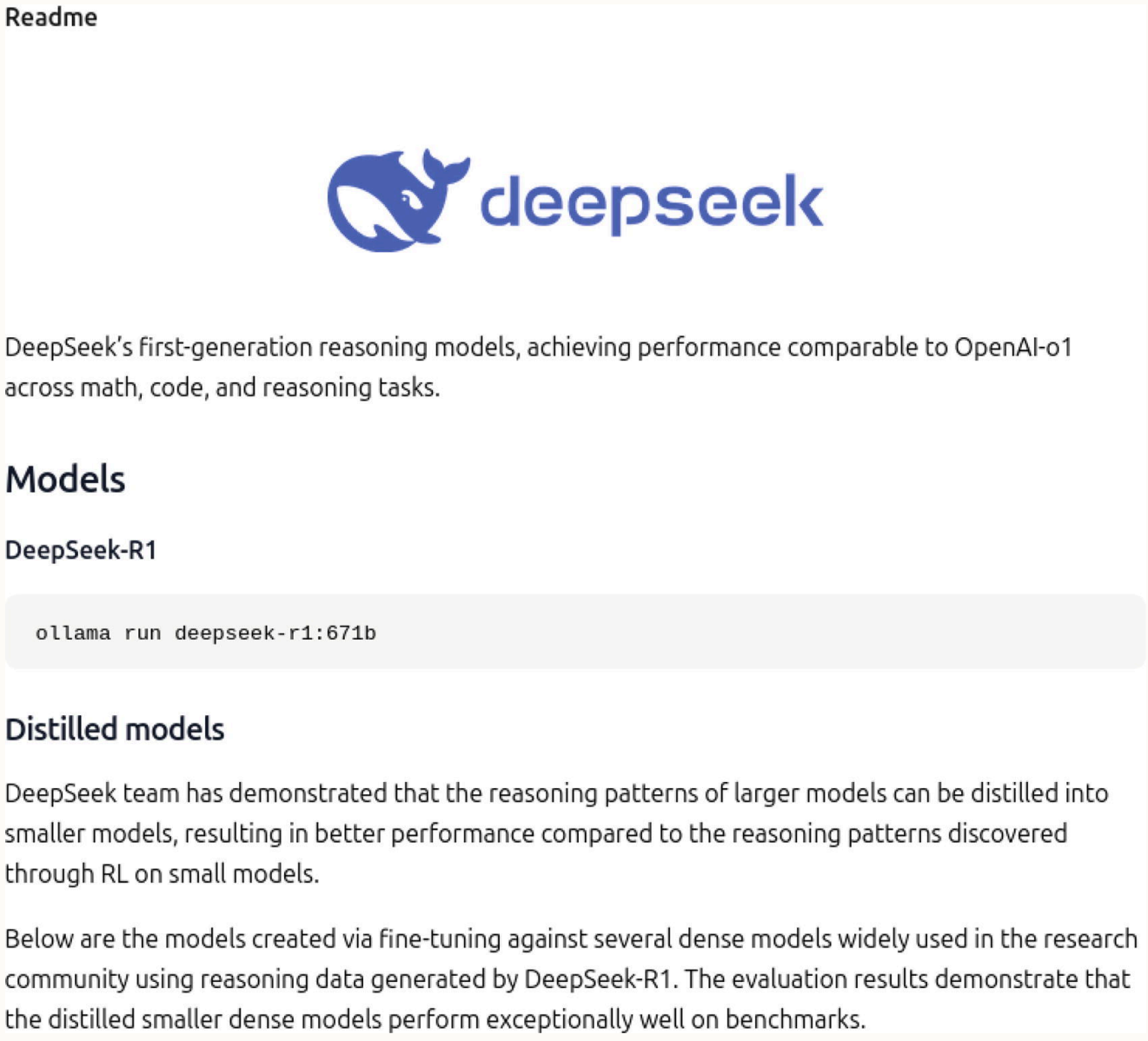


Figure 4-10 Model-specific information

Figure 4-11 shows what different models are available and more documentation, which is always helpful to understand, along with license information. The license information lets you know if it can be used commercially, which is important when building applications.

License

The model weights are licensed under the MIT License. DeepSeek-R1 series support commercial use, allow for any modifications and derivative works, including, but not limited to, distillation for training other LLMs. Please note that:

The Qwen distilled models are derived from Qwen-2.5 series, which are originally licensed under Apache 2.0 License, and now finetuned with 800k samples curated with DeepSeek-R1.

The Llama 8B distilled model is derived from Llama3.1-8B-Base and is originally licensed under llama3.1 license.

The Llama 70B distilled model is derived from Llama3.3-70B-Instruct and is originally licensed under llama3.3 license.

Figure 4-11 License information

Let's copy the command to run 1.5b size model.

```
ollama run deepseek-r1:1.5b
```

Output

```
pulling manifest
pulling aabd4debf0c8... 100% ▕████████████████▏ 1.1 GB
pulling 369ca498f347... 100% ▕████████████████▏  387 B
pulling 6e4c38e1172f... 100% ▕████████████████▏ 1.1 KB
pulling f4d24e9138dd... 100% ▕████████████████▏  148 B
pulling a85fe2a2e58e... 100% ▕████████████████▏  487 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
Type a prompt / query to start interacting with the deepseek model through Ollama by saying a Hello.
>>> Hello
<think>
</think>
Hello! How can I assist you today? 😊
>>> Send a message (/? for help)
```

This specific model is meant for reasoning-based questions. Let's try one.

```
>>> solve u^2−5u−14=0
<think>
Okay, so I have this equation here: \( u^2 - 5u - 14 = 0 \). It's a
quadratic equation in terms of 'u'. Quadratic equations are pretty common
in algebra, and they usually have two solutions. Let me see how to find
those solutions.
First, I remember that for any quadratic equation in the form \( ax^2 + bx
+ c = 0 \), there is a formula called the quadratic formula that can be
used to find the roots:
\[
u = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
\]
In this equation, 'a' is 1 (the coefficient of \( u^2 \)), 'b' is -5 (the
coefficient of 'u'), and 'c' is -14.
Let me plug these values into the quadratic formula:
\[
u = \frac{-(-5) \pm \sqrt{(-5)^2 - 4(1)(-14)}}{2(1)}
\]
Simplifying step by step:
1. Compute \( -b \): \( -(-5) = 5 \)
2. Compute \( b^2 \): \( (-5)^2 = 25 \)
3. Compute \( 4ac \): \( 4 * 1 * (-14) = -56 \)
```

4. Compute the discriminant (\(b^2 - 4ac \)): \(25 - (-56) = 81 \)
Now, plug these back into the formula:
\[
u = \frac{5 \pm \sqrt{81}}{2}
\]
Since \(\sqrt{81} = 9 \), we have:
\[
u = \frac{5 \pm 9}{2}
\]
This gives us two solutions:
1. \(u = \frac{5 + 9}{2} = \frac{14}{2} = 7 \)
2. \(u = \frac{5 - 9}{2} = \frac{-4}{2} = -2 \)
So, the solutions to the equation \(u^2 - 5u - 14 = 0 \) are \(u = 7 \)
and \(u = -2 \).
To verify, let's plug these values back into the original equation:
1. For \(u = 7 \):
\[
(7)^2 - 5(7) - 14 = 49 - 35 - 14 = 0
\]
2. For \(u = -2 \):
\[
(-2)^2 - 5(-2) - 14 = 4 + 10 - 14 = 0
\]
Both values satisfy the equation, so they are indeed the correct
solutions.
In summary, by using the quadratic formula and carefully following each
step, I was able to find that the solutions to \(u^2 - 5u - 14 = 0 \) are
\(u = 7 \) and \(u = -2 \).
</think>
To solve the quadratic equation \(u^2 - 5u - 14 = 0 \), we can use the
quadratic formula:
\[
u = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
\]
Here, \(a = 1 \), \(b = -5 \), and \(c = -14 \).
Plugging in these values:
\[
u = \frac{-(-5) \pm \sqrt{(-5)^2 - 4(1)(-14)}}{2(1)}
\]
\[
u = \frac{5 \pm \sqrt{25 + 56}}{2}
\]
\[
u = \frac{5 \pm \sqrt{81}}{2}
\]
\[
u = \frac{5 \pm 9}{2}
\]
This gives two solutions:
1. \(u = \frac{5 + 9}{2} = 7 \)
2. \(u = \frac{5 - 9}{2} = -2 \)
Verification:
- For \(u = 7 \):
\[
(7)^2 - 5(7) - 14 = 0
\]
\[
49 - 35 - 14 = 0
\]
- For \(u = -2 \):
\[
(-2)^2 - 5(-2) - 14 = 0
\]
\[
4 + 10 - 14 = 0
\]
Both values satisfy the equation, confirming they are correct.
Final Answer:
The solutions to \(u^2 - 5u - 14 = 0 \) are:
\[

```
u = 7 \quad \text{and} \quad u = -2
\]
```

Wow, pretty interesting stuff!

This is so easy to set up and start prompting the other models available for vision and other use cases. You can test with a model and different sizes to see which gives you satisfactory responses specific to your use case. This enables you to experiment in an open-source, efficient, and cost-effective way. Please always keep in mind that Ollama is not recommended to be used to host LLMs in production context but instead is for prototyping and experimenting.

Exploring the APIs Exposed by Ollama

As noted previously, Ollama has exposed an API at http://127.0.0.1:11434. In this section, you will first run a curl command against it to confirm it's available, up, and running.

```
curl http://127.0.0.1:11434
Ollama is running
```

You can find the Ollama documentation at https://github.com/ollama/ollama/tree/main/docs, as shown in Figure 4-12.

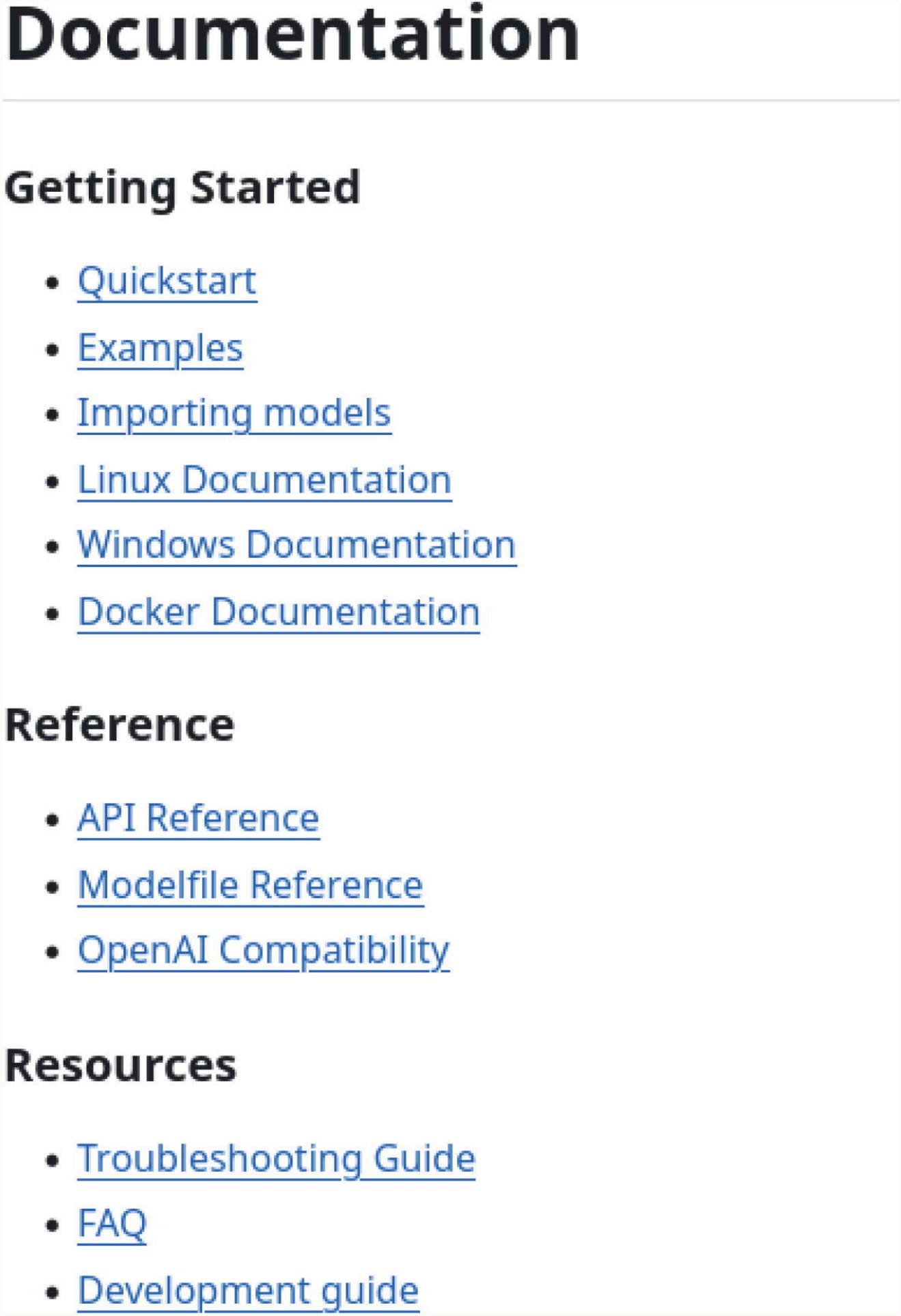


Figure 4-12 Ollama documentation

You can explore the documentation relevant to what you're learning in this section at https://github.com/ollama/ollama/blob/main/docs/api.md (see Figure 4-13).

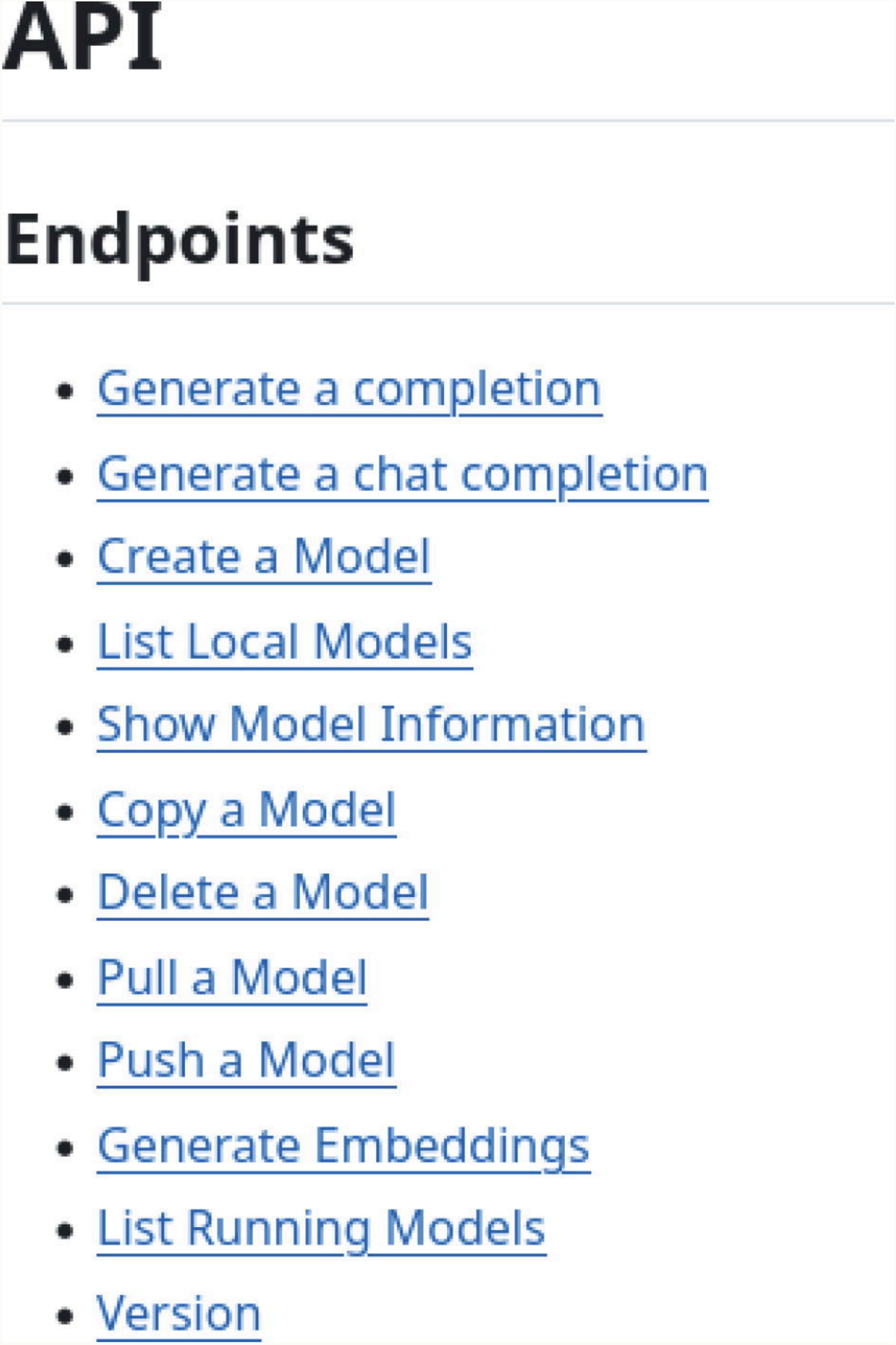


Figure 4-13 Documentation: Ollama API endpoints

Let's generate a completion using the API documentation shown in Figure 4-14. It takes three arguments, namely, the model, which in this case is deepseek-r1:1.5b, and the prompt, which is a query with an option to enable streaming. In this case, you can keep it as false. The output has a response field for the output answer to our query and other fields for duration, count, etc.

Request (No streaming)

Request

A response can be received in one reply when streaming is off.

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false
}'
```

Response

If `stream` is set to `false`, the response will be a single JSON object:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "The sky is blue because it is the color of the sky.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 5043500667,
  "load_duration": 5025959,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 325953000,
  "eval_count": 290,
  "eval_duration": 4709213000
}
```

Figure 4-14 Completion API for text

The response of the query/prompt from DeepSeek can be seen in the following request:

```
curl http://localhost:11434/api/generate -d '{
 "model": "deepseek-r1:1.5b",
 "prompt": "Why is the sky blue?",
 "stream": false
}'
```

Output:

```
{"model":"deepseek-r1:1.5b","created_at":"2025-01-26T15:46:02.889348478Z","response":"\u003cthink\u003e\n\n\u003c/think\u003e\n\nThe sky appearing
```

This is very helpful. You have already built your application centered around the OpenAI APIs. The following is the curl commands, and it does not match the format for the curl for the Ollama APIs.

```
curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
 "model": "gpt-4o-mini",
 "messages": [{"role": "user", "content": "Say this is a test!"}],
 "temperature": 0.7
  }'
```

Not to worry, since OpenAI APIs are quite popular, many LLM hosting vendors similar to Ollama have created abstractions on their API so that your existing APIs to OpenAI still work with a minor adjustment to the URL of the OpenAI you are using. As shown in Figure 4-15 at https://ollama.com/blog/openai-compatibility, Ollama has built-in compatibility with OpenAI APIs. You can review more documentation specific to it at https://github.com/ollama/ollama/blob/main/docs/openai.md.

OpenAI compatibility

February 8, 2024



```
curl http://localhost:11434/v1/chat/completions \
    -H 'Content-Type: application/json' \
    -d '{
      "model": "llama2",
      "messages": [
        {
          "role": "user",
          "content": "Hello!"
        }
      ]
  }'
```

Ollama now has built-in compatibility with the OpenAI Chat Completions API, making it possible to use more tooling and applications with Ollama locally.

Figure 4-15 Ollama OpenAI API compatibility

So let's try this in steps. Our OpenAI API format is as follows:

```
curl https://api.openai.com/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
 "model": "gpt-4o-mini",
 "messages": [{"role": "user", "content": "Say this is a test!"}],
 "temperature": 0.7
 }'
```

Let's convert it to use our Ollama API. As per the documentation, we just need to change the API base URL endpoint, which in our case would be http://localhost:11434. That is all you need to do. You do not need to change anything else in your code. In this case, let's also change the model, since we are using deepseek. gpt-4O-mini is not available to it since it is available internally to OpenAI and is proprietary.

```
curl http://localhost:11434/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer $OPENAI_API_KEY" \
 -d '{
 "model": "deepseek-r1:1.5b",
 "messages": [{"role": "user", "content": "Say this is a test!"}],
 "temperature": 0.7
 }'
```

Output:

```
{
  "id":"chatcmpl-290",
  "object":"chat.completion",
  "created":1737907542,
  "model":"deepseek-r1:1.5b",
  "system_fingerprint":"fp_ollama",
  "choices":[{
    "index":0,
```

```
    "message":{
      "role":"assistant",
      "content":"\u003cthink\u003e\n\n\u003c/think\u003e\n\nYes, it's a test!"
    },
    "finish_reason":"stop"
  }],
  "usage": {
    "prompt_tokens":9,
    "completion_tokens":12,
    "total_tokens":21
  }
}
```

As you can see in Figure 4-16, the JSON response is pretty similar to the output of the OpenAI response.

```
{
    "id": "chatcmpl-abc123",
    "object": "chat.completion",
    "created": 1677858242,
    "model": "gpt-4o-mini",
    "usage": {
        "prompt_tokens": 13,
        "completion_tokens": 7,
        "total_tokens": 20,
        "completion_tokens_details": {
            "reasoning_tokens": 0,
            "accepted_prediction_tokens": 0,
            "rejected_prediction_tokens": 0
        }
    },
    "choices": [
        {
            "message": {
                "role": "assistant",
                "content": "\n\nThis is a test!"
            },
            "logprobs": null,
            "finish_reason": "stop",
            "index": 0
        }
    ]
}
```

Figure 4-16 OpenAI API curl response

Integrating Ollama with Your Existing Application

Let's revisit your Spring Boot application and see what and where you need to make changes to integrate Ollama API so as to have it still be functional and running. First, let's make sure Ollama is installed in the system where you are running your Spring Boot application. Then you need to update the OpenAI API base URL to point to Ollama instead and update from gpt-4o-mini to the deepseek-r1:1.5b model, which can be done in the ChatService.java file.

File: src/main/java/com/example/genai/service/ChatService.java

```
private static final String OPENAI_API_URL = "http://localhost:11434/v1/chat/completions";
public ChatCompletionResponse getChatResponse(String userMessage) {
   RestTemplate restTemplate = new RestTemplate();
  // Build the request body
   ChatCompletionRequest request = new ChatCompletionRequest();
   request.setModel("deepseek-r1:1.5b");
}
```

That should be it. It looks easy enough. Let's run the application and see the output for the same curl command you tried in Chapter 2 with the OpenAI API enabled:

```
curl -X POST "http://localhost:8080/api/chat" -H "Content-Type: application/json" -d '{"userMessage": "Hello!"}'
{"content":"<think>\nAlright, the user just said \"Hello!\" I need to respond in a friendly and welcoming manner. I should let them know I'm here to help with wha
```

Conclusion

This chapter provided a comprehensive guide to self-hosting LLMs using Ollama, offering readers the knowledge and steps required to set up and run these models locally. By exploring the advantages of local hosting—such as enhanced privacy, reduced dependency on external cloud services, and cost-efficiency—we've seen how Ollama simplifies the deployment of language models on local systems. Additionally, the chapter covered the installation process for macOS, Windows, and Linux, along with how to interact with models through Ollama's intuitive API.

When venturing into self-hosting large language models using tools like Ollama, adhering to a set of best practices ensures a smoother, more secure, and more efficient experience. First and foremost, choose the right model for your hardware—balance performance and memory usage with your system's capabilities. Always test smaller models first before scaling to larger ones to assess feasibility. Regularly monitor system resources (CPU, GPU, memory) to prevent unexpected crashes or slowdowns, especially when hosting development machines. Keep Ollama and any related dependencies updated to the latest stable version to benefit from security patches and performance enhancements. If you're exposing APIs, secure them with authentication mechanisms such as API keys or OAuth, and use HTTPS in production environments. For integration, mirror your API contracts with OpenAI's as closely as possible to maintain compatibility and minimize refactoring. While Ollama is ideal for experimentation and prototyping, always evaluate production-readiness by stress-testing the system and planning for scalability, uptime, and long-term support before deploying self-hosted LLMs into live environments.

You also explored how Ollama's API is compatible with OpenAI's, ensuring a smooth transition for developers already working with OpenAI models in their applications. With minimal changes to the API request format, users can switch from OpenAI's cloud service to their own local setup, preserving functionality while cutting down on external dependencies. Finally, the chapter demonstrated the integration process into an existing application, reinforcing how seamless this transition can be with Ollama.

This chapter has set a strong foundation for those looking to experiment with LLMs locally, whether for prototyping or developing applications. While Ollama offers a robust environment for development, it is important to remember that for production-level use, other considerations such as scalability, support, and infrastructure will need to be evaluated. Nonetheless, for anyone looking to explore the world of self-hosting LLMs, Ollama provides an accessible and powerful tool to begin this journey. Let us move on to LangChain.

Previous chapter
< 3. The Transformer Saga—From Attention to Fine-Tuning

Next chapter
5. Power Tools: LangChain4j Quick-Start >